

CHAPTER 3 — Accessing & Traversing Strings

SLIDE 1 — String Indexing

What is String Indexing?

String indexing means using a **position number (index)** to access a particular character in a string.

Think of a string like a row of boxes :

H	E	L	L	O
↑	↑	↑	↑	↑
0	1	2	3	4

 In C, indexing starts from **0**, not 1.

Example

```
char word[] = "HELLO";
```

```
printf("%c", word[0]);
```

Output:

H

Remember

First character → Index 0

 **Memory Trick:**
“C starts counting from ZERO!”

👁️ SLIDE 2 — Accessing Individual Characters

What does “Access a Character” mean?

It means getting a **specific character** from a string using its index.

```
char name[] = "RAVI";
```

Think of it like opening one particular box 📦:

R	A	V	I
0	1	2	3

💻 **Example**

```
printf("%c", name[2]);
```

Output:

V

🔍 **What is happening?**

name[2]

➡ name → string

➡ [2] → position

➡ name[2] → character at position 2

🧠 **Remember**

String + Index = Character

🔄 SLIDE 3 — Using Loops with Strings

Why use a Loop? 🔄

A string can contain many characters.

Instead of accessing them one by one manually, we can use a **loop** to go through them.

Example

```
char word[] = "CODE";
```

```
for(int i = 0; word[i] != '\0'; i++)  
{  
    printf("%c", word[i]);  
}
```

Output:

CODE

What is happening?

i = 0 → C

i = 1 → O

i = 2 → D

i = 3 → E

The loop keeps moving through the string  until it reaches ' **\0** '.

Remember

Loop = Move through characters one by one

SLIDE 4 — Modifying Characters

Can we change a character? 

Yes! We can change an individual character by using its index.

Example

```
char word[] = "HELLO";
```

```
word[1] = 'A';
```

```
printf("%s", word);
```

Output:

HALLO

 **What happened?**

Before:

H E L L O
↑

After:

H A L L O
↑

We changed the character at **index 1**.

 **Important**

For **one character**, use **single quotes**:

```
word[1] = 'A';
```

Not:

```
word[1] = "A"; ❌
```

 **Memory Trick:**

Index → Find the character → Change it



SLIDE 5 — Traversing a String + Recap

What is String Traversing?

Traversing means going through a string **character by character**.

Imagine walking through a row of boxes :

C → O → D → I → N → G
0 1 2 3 4 5

A loop can help us visit each character one by one.

Example

```
char word[] = "CODING";
```

```
for(int i = 0; word[i] != '\0'; i++)  
{  
    printf("%c ", word[i]);  
}
```

Output:

C O D I N G

CHAPTER 3 RECAP

 **Indexing** → Find a character using its position

 **Accessing** → Get an individual character

 **Loop** → Move through characters

 **Modifying** → Change a character

 **Traversing** → Visit characters one by one

BIG MEMORY TRICK

INDEX → ACCESS → LOOP → MODIFY → TRAVERSE

LEVEL CLEARED!

You can now **find, access, change, and move through characters in a string**.

QUESTIONS:-

Q1. MCQ — String Indexing

In C, what is the index of the **first character** in a string?

- ♦ A) 0
- ♦ B) 1
- ♦ C) -1
- ♦ D) 2

 **Hint:** C starts counting positions from zero.

Glossary:

-  **Index** — The position of a character in a string.

 **Answer: A) 0**

Q2. MCQ — Accessing a Character

Consider:

```
char word[] = "HELLO";
```

Which code accesses the character **L** at index **2**?

- ♦ A) `word[1]`
- ♦ B) `word[2]`
- ♦ C) `word[3]`
- ♦ D) `word[4]`

 **Hint:** The first character starts at index 0.

Glossary:

- 👁️ **Access** — To get or use a particular character.
- 📦 **Index** — The position of a character.

✅ **Answer: B) `word[2]`**

✎ Q3. Fill in the Blank — Modifying a Character

Complete the code to change "HELLO" into "HALLO".

```
#include <stdio.h>
```

```
int main()
{
    char word[] = "HELLO";

    word[1] = _____;

    printf("%s", word);

    return 0;
}
```

💡 **Hint:** You are replacing **one character**, so use single quotes.

📖 Glossary:

- ✎ **Modify** — To change something.
- 📦 **Character** — A single letter, number, or symbol.

✅ **Answer: ' A '**

🔄 Q4. Code Choice — Traversing a String

Which code correctly goes through the characters of a string one by one?

```
char word[] = "CODE";
```

♦ **A**

```
for(int i = 0; i < 4; i++)  
    printf("%c", word[i]);
```

♦ **B**

```
for(int i = 1; i < 4; i++)  
    printf("%c", word[i]);
```

♦ **C**

```
for(int i = 0; i < 4; i++)  
    printf("%s", word[i]);
```

♦ **D**

```
for(int i = 0; i <= 4; i++)  
    printf("%c", word[i]);
```

💡 **Hint:** Start from index 0 and use %c for each character.

📖 **Glossary:**

- 🔄 **Traverse** — To go through the characters one by one.
- 🔁 **Loop** — Repeats a block of code.

✅ **Answer: A**

🤖 **Q5. Code Choice — Accessing Characters**

What will be the output?

```
#include <stdio.h>
```

```
int main()  
{  
    char word[] = "CODING";  
  
    printf("%c", word[3]);
```

```
    return 0;  
}
```

- ♦ A) C
- ♦ B) 0
- ♦ C) D
- ♦ D) I

 **Hint:** Write the indexes first:

C	O	D	I	N	G
0	1	2	3	4	5

 **Glossary:**

-  **Index** — The position of a character.
-  **%c** — Used to display one character.

 **Answer: D) I**